

Principi di Progettazione delle Criptovalute

Dispensa Avanzata — Livello LM Informatica

Basata sugli Appunti del Corso di Francesco Pasquale

Università degli Studi di Roma "Tor Vergata", A.A. 2025/26

Elaborazione e Ampliamento Accademico

Anno Accademico 2025–2026

Prefazione

Questa dispensa costituisce un’elaborazione avanzata degli appunti del corso *Principles of Cryptocurrency Design*, tenuto dal Prof. Francesco Pasquale. L’obiettivo è fornire uno strumento di studio completo per studenti di livello magistrale e dottorandi, integrando le nozioni di base con approfondimenti teorici, contesto storico, dimostrazioni rigorose e connessioni interdisciplinari. Il filo conduttore è la domanda fondamentale: *come è possibile costruire un sistema di consenso distribuito robusto in assenza di un’autorità centralizzata?* La risposta a questa domanda, elaborata nel corso di decenni di ricerca in informatica teorica e crittografia, ha portato alla nascita delle moderne criptovalute.

Indice

1	Introduzione: Il Problema del Consenso Distribuito	4
1.1	Motivazione e Contesto Storico	4
1.2	La Blockchain come Struttura Dati Distribuita	4
2	Il Problema Byzantine Broadcast	4
2.1	Definizione Formale	5
2.2	Analisi delle Proprietà Richieste	5
3	Modelli di Calcolo Distribuito	5
3.1	Le Tre Dimensioni del Modello	5
3.1.1	Modello Sincrono vs Asincrono	6
3.1.2	Modello Autenticato vs Non Autenticato	6
3.1.3	Modello Permissioned vs Permissionless	6
3.2	La Gerarchia dei Modelli	7
4	Il Modello Sincrono con PKI: Il Protocollo di Dolev-Strong	7
4.1	Contesto e Motivazione	7
4.2	Il Protocollo di Dolev-Strong	7
4.3	Correttezza del Protocollo di Dolev-Strong	8

5	Schemi di Firme Digitali	8
5.1	Definizione Formale	8
5.2	Sicurezza: Existential Unforgeability under Chosen Message Attack	9
5.3	Lo Schema RSA con Hash: Hashed RSA	9
5.3.1	Costruzione	9
5.3.2	Correttezza dell’RSA	9
5.3.3	Il Pericolo del Textbook RSA	10
5.4	Il Test di Primalità di Fermat	10
6	Lower Bound per Byzantine Broadcast senza PKI	10
6.1	Il Teorema di Impossibilità	11
6.2	La Dimostrazione: Tecnica della Simulazione	11
7	Un Protocollo Probabilistico per Byzantine Broadcast (Sincrono)	12
7.1	Motivazione: Superare il Lower Bound con la Randomizzazione	12
7.2	Il Protocollo Randomizzato Sincrono	12
8	Byzantine Agreement	13
8.1	Definizione e Differenze con Byzantine Broadcast	13
9	Il Modello Asincrono e il Teorema FLP	14
9.1	Il Modello Asincrono	14
9.2	Il Teorema FLP: Enunciato	14
9.3	Struttura della Dimostrazione	14
9.3.1	Lemma 1: Esistenza di una Configurazione Iniziale Bivalente	15
9.3.2	Lemma 2: Mantenimento della Bivalenza	15
9.3.3	Conclusione del Teorema FLP	15
10	Un Protocollo Probabilistico per BA nel Modello Asincrono	16
10.1	Il Protocollo di Ben-Or (1983)	16
11	Il Protocollo di Consenso di Bitcoin: Nakamoto Consensus	17
11.1	Il Cambiamento di Paradigma: da Permissioned a Permissionless	17
11.2	Proof-of-Work: Definizione e Funzione	17
11.3	Struttura della Blockchain di Bitcoin	18
11.3.1	La Struttura dell’Header	18
11.3.2	Le Proprietà di Eventual Consistency e Liveness	18
11.4	Il Meccanismo di Aggiustamento del Target	18
11.5	La Regola della Catena Più Lunga	19
12	Le Transazioni Bitcoin	19
12.1	Struttura di Base	19
12.2	Il Modello UTXO	20
12.3	Transazione Coinbase e Creazione di Bitcoin	20
12.4	Validità di una Transazione	20
13	Il Linguaggio Script di Bitcoin	21
13.1	Architettura del Linguaggio	21
13.2	Tipi Principali di Locking Script	21
13.2.1	Pay-to-Public-Key (P2PK)	21
13.2.2	Pay-to-Public-Key-Hash (P2PKH)	21
13.2.3	Pay-to-Multisig (P2MS)	22

13.2.4 Pay-to-Script-Hash (P2SH)	22
13.2.5 OP_RETURN e Dati nella Blockchain	22
14 Crittografia basata su Curve Ellittiche (ECC)	22
14.1 Definizione e Proprietà Matematiche	22
14.2 Campi Finiti e la Curva SECP256K1	23
14.3 Somma di Punti e Moltiplicazione Scalare	23
14.4 Discrete Logarithm Problem e ECDH	23
15 Sintesi e Connessioni Interdisciplinari	24
15.1 Il Quadro d'Insieme	24
15.2 Connessioni con Altre Discipline	24
Riferimenti Bibliografici	25

1 Introduzione: Il Problema del Consenso Distribuito

1.1 Motivazione e Contesto Storico

Il problema di far concordare entità distribuite su un valore comune è uno dei problemi fondamentali dell'informatica distribuita. La sua rilevanza va ben al di là delle criptovalute: sistemi di controllo aeronautico, basi di dati replicate, sistemi di voto elettronico e reti peer-to-peer richiedono tutti forme di consenso distribuito.

Nota Storica — Le origini del problema del consenso

Il problema del consenso in presenza di agenti non affidabili fu formulato per la prima volta in forma precisa da Lamport, Shostak e Pease nel loro celebre articolo del 1982, *The Byzantine Generals Problem*. La metafora militare usata nell'articolo è elegante: un gruppo di generali dell'esercito bizantino, che comunicano solo tramite messaggeri, devono accordarsi su un piano d'azione comune (attaccare o ritirarsi), pur sapendo che alcuni di loro potrebbero essere traditori. Il termine *Bizantino* è diventato sinonimo di "comportamento arbitrario e potenzialmente malevolo" nella letteratura informatica.

Negli anni '80, lo studio di questi problemi era motivato da applicazioni come sistemi ridondanti per applicazioni critiche (ad es. controllo di volo) e basi di dati distribuite. Il cambiamento di paradigma introdotto da Bitcoin nel 2008 ha riportato questi problemi al centro dell'attenzione, aggiungendo la dimensione del modello *permissionless* e dell'incentivazione economica.

1.2 La Blockchain come Struttura Dati Distribuita

Prima di entrare nei dettagli tecnici, è utile posizionare il problema nel contesto delle moderne criptovalute. Il risultato finale a cui il corso tende è la comprensione del protocollo di consenso di Nakamoto (Bitcoin), che affronta il problema del consenso in un contesto radicalmente diverso da quello degli anni '80:

- **Ambiente aperto (permissionless):** chiunque può partecipare senza autorizzazione;
- **Scalabilità:** il sistema deve funzionare con decine di migliaia di nodi;
- **Incentivi economici:** i partecipanti hanno un interesse economico a comportarsi correttamente;
- **Robustezza computazionale:** il consenso si basa su assunzioni di difficoltà computazionale (proof-of-work) anziché su assunzioni strutturali sui canali di comunicazione.

Il percorso didattico segue un arco logico preciso: si parte dai risultati fondamentali degli anni '80 (modello sincrono, poi asincrono), si dimostrano i limiti intrinseci di questi modelli, e si mostra come Bitcoin abbia adottato un approccio diverso per superare tali limiti.

2 Il Problema Byzantine Broadcast

2.1 Definizione Formale

Definizione Formale — Byzantine Broadcast (BB)

Sia dato un sistema distribuito con n nodi, etichettati $[n] = \{1, 2, \dots, n\}$. Ogni nodo può:

- eseguire computazioni locali arbitrarie;
- inviare messaggi di lunghezza arbitraria a qualunque altro nodo.

Tra gli n nodi, f sono **corrotti** ($0 \leq f \leq n$) e gli altri $n - f$ sono **onesti**. Il nodo 1 (la *sorgente*) riceve in input un messaggio b . Il problema **Byzantine Broadcast** richiede la progettazione di un protocollo al termine del quale ogni nodo onesto i produca un output y_i tale che:

1. **Consistency**: Per ogni coppia di nodi onesti i e j , $y_i = y_j$;
2. **Validity**: Se la sorgente è onesta, allora $y_i = b$ per ogni nodo onesto i .

I nodi corrotti non applicano il protocollo: nella nostra analisi worst-case, assumiamo che tutti i nodi corrotti si coalizzino per far fallire il protocollo, e che non sappiamo a priori quali siano.

2.2 Analisi delle Proprietà Richieste

È importante comprendere il significato delle due proprietà in relazione:

- **Consistency senza Validity** è banale: tutti i nodi onesti possono semplicemente restituire 0, ignorando la sorgente.
- **Validity senza Consistency** è anch'essa banale: ogni nodo onesto restituisce il messaggio ricevuto dalla sorgente (o 0 se non ne ha ricevuto).
- La difficoltà sta nel garantirle *entrambe contemporaneamente* in presenza di nodi corrotti.

Esempio Applicativo — Perché il BB è difficile

Consideriamo $n = 3$ nodi: A (sorgente), B, C. Supponiamo che A sia corrotto e invii il bit 1 a B e il bit 0 a C. B e C non hanno modo di sapere cosa ha detto A all'altro. Se B e C si fidano di quello che ha detto loro A:

- B restituisce 1 (soddisfazione di validity locale per B);
- C restituisce 0 (soddisfazione di validity locale per C);

La consistency è violata. Questo esempio mostra perché è necessario un protocollo di comunicazione multi-round.

3 Modelli di Calcolo Distribuito

3.1 Le Tre Dimensioni del Modello

La progettazione e l'analisi di protocolli distribuiti dipende criticamente dal modello adottato. Tre dimensioni principali caratterizzano il modello:

3.1.1 Modello Sincrono vs Asincrono

Definizione — Modelli di Sincronizzazione

- **Modello Sincrono:** Esiste un *global clock* discreto, noto a tutti i nodi, che scandisce i *round*. Ogni messaggio inviato nel round t arriva a destinazione prima dell'inizio del round $t + 1$. Ogni round ha una durata massima nota.
- **Modello Asincrono:** Non esiste un clock globale; i messaggi vengono recapitati in tempo arbitrario (ma finito). Non è possibile distinguere un nodo lento da un nodo corrotto che non risponde.
- **Modello Parzialmente Sincrono:** Modello intermedio in cui esistono bound sui tempi, ma non sono noti a priori ai nodi (noti solo all'analista), oppure valgono solo dopo un certo momento *GST* (Global Stabilization Time).

La differenza tra questi modelli ha implicazioni drammatiche sulla trattabilità del consenso: vedremo che nel modello asincrono il consenso deterministico è *impossibile* anche con un solo nodo corrotto (teorema FLP).

3.1.2 Modello Autenticato vs Non Autenticato

Definizione — Autenticazione e PKI

- **Modello Non Autenticato:** Quando un nodo j riceve un messaggio apparentemente da i , non può verificare che il mittente sia effettivamente i . Un nodo corrotto può impersonare qualunque altro nodo.
- **Modello Autenticato (con PKI):** Esiste una *Public Key Infrastructure* setup: ogni nodo i ha una coppia di chiavi (sk_i, pk_i) ; l'insieme $\{pk_i : i \in [n]\}$ è *common knowledge*. Un nodo può firmare messaggi con la propria chiave segreta, e chiunque può verificare la firma con la corrispondente chiave pubblica. Indichiamo con $\langle m \rangle_i$ il messaggio m firmato da i .

L'esistenza di una PKI è un'assunzione molto forte: richiede la presenza di un'autorità di certificazione fidata, o di un setup iniziale sicuro tra i partecipanti.

3.1.3 Modello Permissioned vs Permissionless

Definizione — Permissioned e Permissionless

- **Modello Permissioned:** L'insieme degli n nodi è fissato all'inizio del protocollo ed è noto a tutti i partecipanti. Tutti i risultati degli anni '80 operano in questo modello.
- **Modello Permissionless:** Nuovi nodi possono aggregarsi o abbandonare la rete in qualunque momento. L'insieme dei partecipanti non è noto e varia dinamicamente. Bitcoin e le principali criptovalute moderne operano in questo modello.

Il passaggio da permissioned a permissionless introduce il problema dei *Sybil attack*: un singolo agente può creare arbitrariamente molte identità per dominare il sistema.

3.2 La Gerarchia dei Modelli

Il modello più favorevole — sincrono, autenticato, permissioned — è quello in cui è più facile progettare protocolli efficienti. Man mano che si rimuovono assunzioni, aumenta la difficoltà:

$$\underbrace{\text{Sincrono} + \text{PKI} + \text{Permissioned}}_{\text{più favorevole}} \longrightarrow \underbrace{\text{Asincrono} + \text{No PKI} + \text{Permissionless}}_{\text{meno favorevole}}$$

4 Il Modello Sincrono con PKI: Il Protocollo di Dolev-Strong

4.1 Contesto e Motivazione

Partiamo dalla configurazione più favorevole: modello sincrono, PKI setup, permissioned. In questo contesto, il problema del Byzantine Broadcast è risolvibile per qualunque numero f di nodi corrotti. Il protocollo di Dolev-Strong (1983) realizza questa possibilità in modo elegante.

Nota Storica — Dolev e Strong (1983)

Danny Dolev e H. Raymond Strong pubblicarono nel 1983 il loro protocollo su *SIAM Journal on Computing*, fornendo il primo protocollo deterministico per il Byzantine Broadcast che funzionasse per qualunque numero di nodi corrotti nel modello sincrono autenticato. La chiave dell'approccio è l'utilizzo delle firme digitali per rendere “non ripudiabile” la comunicazione: un nodo corrotto non può forgiare la firma di un nodo onesto, e quindi non può mentire su ciò che un nodo onesto ha detto.

4.2 Il Protocollo di Dolev-Strong

Protocollo di Dolev-Strong per Byzantine Broadcast

Inizializzazione (Round 0): Ogni nodo u inizializza un insieme $E_u = \emptyset$. La sorgente (nodo 1) riceve in input b e invia $\langle b \rangle_1$ a tutti i nodi.

Per ogni Round $r = 1, \dots, f$: Ogni nodo u esegue:

1. Per ogni messaggio $\langle \hat{b} \rangle_{1, \sigma_1, \sigma_2, \dots, \sigma_{r-1}}$ con r firme distinte (inclusa quella della sorgente) ricevuto nel Round $r - 1$:
2. Se $\hat{b} \notin E_u$:
 - (a) Aggiungi $\hat{b}$ a E_u ;
 - (b) Firma il messaggio e invia $\langle \hat{b} \rangle_{1, \sigma_1, \dots, \sigma_{r-1}, \sigma_u}$ a tutti i nodi.

Round $f + 1$ (Output): Ogni nodo u :

1. Per ogni messaggio con $f + 1$ firme distinte (inclusa quella della sorgente) ricevuto nel Round f : se $\hat{b} \notin E_u$, aggiunge $\hat{b}$ a E_u .
2. Se $|E_u| = 1$: restituisce in output l'unico elemento di E_u .
3. Altrimenti: restituisce in output 0.

4.3 Correttezza del Protocollo di Dolev-Strong

Teorema — Correttezza di Dolev-Strong

Nel modello sincrono con PKI setup, il protocollo di Dolev-Strong soddisfa **Validity** e **Consistency** per qualunque numero f di nodi corrotti, in $f + 1$ round di comunicazione.

Dimostrazione (Sketch). **Validity:** Supponiamo che la sorgente (nodo 1) sia onesta con input b . Nel Round 0, la sorgente invia $\langle b \rangle_1$ a tutti. Ogni nodo onesto riceve $\langle b \rangle_1$ e aggiunge b al proprio insieme. Nei round successivi, i nodi corretti continuano a propagare b con firme aggiuntive. Non possono introdurre un valore diverso da b con la firma della sorgente, perché non possono forgiare $\langle \hat{b} \rangle_1$ per $\hat{b} \neq b$. Quindi $E_u = \{b\}$ per ogni nodo onesto u , e l'output è b .

Consistency: Dobbiamo mostrare che se un nodo onesto u aggiunge $\hat{b}$ al suo insieme E_u nel Round r , allora anche ogni altro nodo onesto v aggiungerà $\hat{b}$ al suo insieme entro il Round $r + 1$.

Per $r \leq f$: se u riceve nel Round $r - 1$ il messaggio $\langle \hat{b} \rangle_{1, \sigma_1, \dots, \sigma_{r-1}}$ con r firme distinte (inclusa quella della sorgente), e aggiunge $\hat{b}$ a E_u , allora u firma e invia il messaggio con $r + 1$ firme distinte a tutti. Quindi v riceve nel Round r un messaggio con $r + 1 \geq 2$ firme distinte e aggiunge $\hat{b}$ a E_v (se non l'aveva già fatto). Nel Round f : v riceve un messaggio con $f + 1$ firme e aggiunge $\hat{b}$ a E_v .

Quindi: se u ha $\hat{b} \in E_u$ al termine, allora $\hat{b} \in E_v$ per ogni nodo onesto v . Analogamente per v . Quindi $E_u = E_v$ per tutti i nodi onesti, e l'output è lo stesso per tutti. $\square$

Approfondimento — Il Ruolo delle Firme Digitali

Il meccanismo chiave che rende funzionante il protocollo di Dolev-Strong è la **non falsificabilità** delle firme digitali. Se un nodo corrotto riceve $\langle b \rangle_1$ e vuole far credere a qualcuno che la sorgente abbia inviato $\langle b' \rangle_1$ per $b' \neq b$, non può farlo: non conosce la chiave segreta sk_1 della sorgente.

Questo crea un meccanismo di “prove di autenticità” a catena: un messaggio con r firme distinte costituisce una prova che r nodi distinti hanno ricevuto quel valore dalla sorgente e lo hanno ritrasmesso. Un nodo corrotto può firmare messaggi falsi col proprio identificatore, ma non può impostare il contatore di firme distinte oltre il numero di nodi corrotti che controlla.

Senza PKI, come vedremo nella prossima sezione, questo meccanismo di prova non è disponibile, e l'impossibilità diventa la regola per $f \geq n/3$.

5 Schemi di Firme Digitali

5.1 Definizione Formale

Definizione Formale — Schema di Firma Digitale

Uno **schema di firma digitale** è una tripla di algoritmi probabilistici polinomiali (KeyGen, Sign, Verify):

- $\text{KeyGen}(1^\lambda) = (sk, pk)$: prende in input il *security parameter* $\lambda \in \mathbb{N}$ (in unario 1^λ) e restituisce una coppia di chiavi $sk \in \{0, 1\}^{p(\lambda)}$ e $pk \in \{0, 1\}^{p(\lambda)}$, dove p è un polinomio.
- $\text{Sign}(sk, m) = \sigma$: prende la chiave segreta e un messaggio $m \in \{0, 1\}^*$; restituisce una firma $\sigma \in \{0, 1\}^{p(\lambda)}$.
- $\text{Verify}(m, pk, \sigma) \in \{\text{True}, \text{False}\}$: verifica se σ è una firma valida per m rispetto a pk .

Correttezza: Per ogni λ , ogni $(sk, pk) \leftarrow \text{KeyGen}(1^\lambda)$, ogni $m \in \{0, 1\}^*$:

$$\text{Verify}(m, pk, \text{Sign}(sk, m)) = \text{True}$$

5.2 Sicurezza: Existential Unforgeability under Chosen Message Attack

Definizione — Sicurezza (EU-CMA)

Lo schema è **sicuro** (EU-CMA: Existential Unforgeability under Chosen Message Attack) se per ogni algoritmo probabilistico polinomiale $\mathcal{A}$ (l'avversario) che:

1. riceve pk in input;
2. può fare un numero polinomiale di query alla funzione $\text{Sign}(sk, \cdot)$, ottenendo coppie $(\hat{m}, \hat{\sigma})$;
3. restituisce in output una coppia (m, σ) ;

la probabilità che $\text{Verify}(m, pk, \sigma) = \text{True}$ e $m \notin \{\hat{m}_1, \hat{m}_2, \dots\}$ (i.e., m non era uno dei messaggi interrogati) è $\text{negl}(\lambda)$.

Una funzione $f : \mathbb{N} \rightarrow [0, 1]$ è **negligible** se per ogni polinomio p esiste n_0 tale che $f(n) < 1/p(n)$ per ogni $n \geq n_0$.

5.3 Lo Schema RSA con Hash: Hashed RSA

5.3.1 Costruzione

Esempio Applicativo — Schema Hashed RSA

Lo schema RSA di firma digitale funziona come segue:

- **KeyGen**(1^λ): genera due numeri primi p, q dell'ordine di λ bit; calcola $n = pq$; genera e coprimo con $(p-1)(q-1)$; calcola d tale che $de \equiv 1 \pmod{(p-1)(q-1)}$. Restituisce $pk = (n, e)$, $sk = (n, d)$.
- **Sign**(sk, m): calcola $h = \text{Hash}(m)$; restituisce $h^d \bmod n$.
- **Verify**(m, pk, σ): calcola $h = \text{Hash}(m)$; verifica se $\sigma^e \equiv h \pmod{n}$.

5.3.2 Correttezza dell'RSA

La correttezza segue dal **Teorema di Eulero-Fermat**. Poiché $de \equiv 1 \pmod{(p-1)(q-1)}$, si ha $de = 1 + k(p-1)(q-1)$ per qualche intero k . Per il Teorema di Fermat:

$$h^{de} = h \cdot (h^{(p-1)(q-1)})^k \equiv h \cdot 1^k = h \pmod{n}$$

(quando $\gcd(h, n) = 1$). Quindi $\text{Verify}(m, pk, \text{Sign}(sk, m)) = \text{True}$.

5.3.3 Il Pericolo del Textbook RSA

Attenzione — Vulnerabilità del Textbook RSA

Se si firma direttamente il messaggio (senza hasharlo), lo schema è **insicuro**. Un avversario può:

1. Scegliere una firma arbitraria $\sigma_{\text{fake}} \in \mathbb{Z}_n$;
2. Calcolare $m_{\text{fake}} = \sigma_{\text{fake}}^e \bmod n$;
3. Notare che $\text{Verify}(m_{\text{fake}}, \text{pk}, \sigma_{\text{fake}}) = \text{True}$, poiché $\sigma_{\text{fake}}^e \equiv m_{\text{fake}} \pmod{n}$.

Questo è un attacco di *falsificazione esistenziale*: l'avversario produce una coppia $(m_{\text{fake}}, \sigma_{\text{fake}})$ valida senza conoscere sk . L'utilizzo di una funzione hash crittografica risolve il problema: m_{fake} sarebbe un intero casuale senza significato semantico, e l'avversario non può controllarne il preimage rispetto alla funzione hash.

5.4 Il Test di Primalità di Fermat

Un aspetto tecnico rilevante per la generazione delle chiavi RSA è la verifica che p e q siano primi. Il seguente teorema è fondamentale:

Teorema di Fermat (1640)

Se $p \in \mathbb{N}$ è un numero primo, allora $a^{p-1} \equiv 1 \pmod{p}$ per ogni $a \in \{1, \dots, p-1\}$.

Dimostrazione. Sia $S = \{1, 2, \dots, p-1\}$ e $aS = \{a \bmod p, 2a \bmod p, \dots, (p-1)a \bmod p\}$. Poiché p è primo e $\gcd(a, p) = 1$, le moltiplicazioni per a modulo p sono una permutazione di S , quindi $S = aS$. Di conseguenza:

$$\prod_{x \in S} x \equiv \prod_{x \in aS} x = a^{p-1} \prod_{x \in S} x \pmod{p}$$

Poiché $\prod_{x \in S} x = (p-1)!$ è invertibile modulo p (essendo p primo), possiamo dividere entrambi i membri per esso, ottenendo $a^{p-1} \equiv 1 \pmod{p}$. $\square$

Approfondimento — Numeri di Carmichael e i Limiti del Test di Fermat

Il test di primalità di Fermat è il seguente: per verificare se n è primo, si scelgono t valori casuali $a \in \{2, \dots, n-2\}$ e si verifica se $a^{n-1} \equiv 1 \pmod{n}$. Se il test fallisce per qualche a , n è certamente composto. Se il test passa per tutti i t valori, n è probabilmente primo. Tuttavia, esistono i **numeri di Carmichael**: numeri composti n tali che $a^{n-1} \equiv 1 \pmod{n}$ per tutti gli a con $\gcd(a, n) = 1$. Per tali numeri, il test di Fermat non può rilevare la composizione. I primi sette numeri di Carmichael sono: 561, 1105, 1729, 2465, 2821, 6601, 8911.

Per questo motivo, in pratica si utilizzano test di primalità più robusti, come il **test di Miller-Rabin**, che non ha falsi positivi per numeri di Carmichael.

Un risultato chiave: se n è composto e *non* è un numero di Carmichael, allora l'insieme $W_n = \{a \in \{1, \dots, n-1\} : a^{n-1} \not\equiv 1 \pmod{n}\}$ soddisfa $|W_n| \geq n/2$. Questo garantisce che il test di Fermat abbia probabilità di errore al più $(1/2)^t$ su tali numeri.

6 Lower Bound per Byzantine Broadcast senza PKI

6.1 Il Teorema di Impossibilità

Quando si rimuove l'assunzione di PKI, la situazione cambia drasticamente: nessun protocollo può garantire simultaneamente Validity e Consistency se ci sono $f \geq n/3$ nodi corrotti.

Teorema — Impossibilità BB senza PKI ($f \geq n/3$)

In assenza di PKI setup e con $f \geq n/3$ nodi corrotti, **nessun protocollo** per il problema Byzantine Broadcast può garantire Validity e Consistency.
(Pease, Shostak, Lamport, 1980; Fischer, Lynch, Merritt, 1985)

6.2 La Dimostrazione: Tecnica della Simulazione

La dimostrazione usa un argomento di impossibilità per assurdo, basato sulla **tecnica di simulazione/indistinguibilità**.

Dimostrazione. Consideriamo il sistema G con 3 nodi: A (sorgente), B, C. Supponiamo per assurdo che esista un protocollo Π che garantisce Validity e Consistency anche in presenza di 1 nodo corrotto.

Costruiamo l'esperimento concettuale H in Figura ??, un **anello** di 6 nodi in cui ogni nodo comunica solo con i suoi due vicini:

- x e x' sono copie esatte di A (con la stessa chiave privata, nel caso autenticato);
- y e y' sono copie esatte di B;
- z e z' sono copie esatte di C.

Osservazione chiave: Nessun nodo può distinguere se si trova in G o in H . Per esempio, z' in H comunica con x (una copia di A) e y' (una copia di B), ricevendo esattamente gli stessi messaggi che riceverebbe C in G .

Ora consideriamo l'esecuzione del protocollo Π in H dove x riceve input 1 e x' riceve input 0.

Claim 1: x e y devono dare in output 1.

Dimostrazione: In G , se A e B sono onesti e C è corrotto, l'esecuzione di Π garantisce che A e B diano output 1 (A è onesta con input 1, per Validity). Ora, C corrotto in G potrebbe comportarsi esattamente come z e z' si comporterebbero in H nella simulazione, inviando ad A e B i rispettivi messaggi che ricevono da z' e z . Dunque x e y in H ricevono gli stessi messaggi di A e B in G , e quindi danno output 1.

Claim 2: x' e z devono dare in output 0.

Dimostrazione: Simmetrica al Claim 1, considerando G con input 0 e B come corrotto.

Claim 3: y e z devono dare in output lo stesso bit.

Dimostrazione: In G , se B e C sono onesti e A è corrotta, per Consistency B e C devono dare lo stesso output.

I tre Claim si contraddicono: dal Claim 1, y output 1; dal Claim 2, z output 0; ma dal Claim 3, y e z devono dare lo stesso output. Assurdo. $\square$

Approfondimento — Perché Dolev-Strong non Contraddice il Lower Bound

Il teorema di impossibilità si applica in assenza di PKI. La chiave della dimostrazione è che, nell'esperimento H , i nodi x e x' sono "copie esatte" di A: hanno lo stesso stato, lo stesso programma, lo stesso input locale. In un sistema con PKI, invece, la stessa chiave privata non può essere usata da due nodi distinti senza che questo sia rilevabile dagli altri nodi: le firme permettono ai nodi di rilevare che un messaggio proviene da un'identità specifica. Quindi, nell'esperimento H con PKI, i nodi y e z potrebbero ricevere messaggi firmati sia

da x che da x' con la stessa chiave di A , il che è un evento impossibile nel sistema reale G (A è un solo nodo). Il protocollo può quindi rilevare l'inconsistenza e agire di conseguenza.

7 Un Protocollo Probabilistico per Byzantine Broadcast (Sincrono)

7.1 Motivazione: Superare il Lower Bound con la Randomizzazione

Il lower bound precedente riguarda protocolli deterministici in assenza di PKI. Aggiungendo la *randomizzazione* è possibile superarlo parzialmente: per $f < n/3$, esiste un protocollo probabilistico che soddisfa Validity con probabilità 1 e Consistency con alta probabilità.

Definizione — High Probability (w.h.p.)

Data una successione di eventi $\{E_n\}_n$, diciamo che E_n avviene **con alta probabilità** (w.h.p., *with high probability*) se esistono $n_0 \in \mathbb{N}$ e $c > 0$ tali che $\mathbb{P}(E_n) \geq 1 - n^{-c}$ per ogni $n \geq n_0$.

7.2 Il Protocollo Randomizzato Sincrono

Il protocollo utilizza un **random oracle** $H : \mathbb{N} \cup \{0\} \rightarrow [n]$ con $H(0) = 1$ (il nodo sorgente è sempre il leader della prima iterazione).

Protocollo Sincrono Randomizzato per BB ($f < n/3$)

Ogni iterazione $t = 0, \dots, k-1$ comprende tre round:

1. **Round $3t$ (Leader):** Si seleziona il leader $\ell_t = H(t)$. Se $sb_{\ell_t} \neq \perp$, ℓ_t invia sb_{ℓ_t} a tutti; altrimenti sceglie un bit in $\{0, 1\}$ uniformemente a caso e lo invia.
2. **Round $3t + 1$ (Propagazione):** Ogni nodo u : se $sb_u \neq \perp$, invia sb_u a tutti; altrimenti invia il bit ricevuto dal leader.
3. **Round $3t + 2$ (Votazione):** Ogni nodo u : se esiste $\hat{b}$ ricevuto da almeno $2n/3$ nodi distinti, imposta $sb_u = \hat{b}$; altrimenti imposta $sb_u = \perp$.

Output: Dopo k iterazioni, ogni nodo u restituisce sb_u .

Teorema — Correttezza del Protocollo Randomizzato

Se il numero di nodi corrotti è $f < n/3$, il protocollo soddisfa Validity con probabilità 1 e Consistency con probabilità almeno $1 - (2/3)^k$.

Dimostrazione (Sketch). **Validity:** Se la sorgente è onesta, nell'iterazione $t = 0$ tutti i nodi onesti ricevono b dalla sorgente (leader $\ell_0 = H(0) = 1$) e lo inviano a tutti. Poiché ci sono almeno $n - f > 2n/3$ nodi onesti, ogni nodo onesto riceve b da almeno $2n/3$ nodi e fissa $sb_u = b$. Nelle iterazioni successive, i nodi onesti continuano a propagare b , mantenendo $sb_u = b$.

Consistency: La dimostrazione si articola in cinque passi:

1. *Non ambiguità:* In ogni iterazione t , non possono esistere due nodi onesti u, v con $sb_u = 0$ e $sb_v = 1$ al termine dell'iterazione. (Questo perché $2n/3 > n/2 + f$, quindi due maggioranze diverse non possono coesistere.)
2. *Iterazione fortunata:* Un'iterazione t è *lucky* se: (i) il leader ℓ_t è onesto, e (ii) il bit inviato da ℓ_t coincide con il valore sb di tutti i nodi onesti che hanno $sb \neq \perp$ all'inizio dell'iterazione.

3. *Implicazione*: Se esiste un'iterazione lucky, tutti i nodi onesti fissano lo stesso valore e lo mantengono.
4. *Probabilità di luck*: Qualunque cosa sia successa in precedenza, un'iterazione è lucky con probabilità almeno $1/3$.
5. *Conclusione*: La probabilità che nessuna delle k iterazioni sia lucky è al più $(2/3)^k$.

□

Approfondimento — Il Random Oracle e la Soglia $2n/3$

Il random oracle: La funzione H viene modellata come un *random oracle* nella dimostrazione: per ogni input t , $H(t)$ si comporta come un valore scelto uniformemente a caso in $[n]$, indipendentemente da tutti gli altri. Nella pratica, si usa una funzione hash crittografica. L'idea è che i nodi corrotti non possano prevedere chi sarà il prossimo leader, rendendo più difficile coordinarsi per attaccare l'iterazione fortunata.

La soglia $2n/3$: Perché $2n/3$ e non $n/2$? Con $f < n/3$ nodi corrotti e $n - f > 2n/3$ nodi onesti, se un nodo onesto riceve $\hat{b}$ da $\geq 2n/3$ nodi, almeno $2n/3 - f > n/3$ di questi sono onesti. Questo garantisce che l'informazione sia genuina. Se si usasse la soglia $h < 2n/3$, i nodi corrotti potrebbero far credere a due nodi onesti che la maggioranza abbia votato per valori diversi. Se si usasse $h > 2n/3$, la soglia potrebbe non essere raggiungibile con f nodi corrotti che non partecipano.

8 Byzantine Agreement

8.1 Definizione e Differenze con Byzantine Broadcast

Definizione Formale — Byzantine Agreement (BA)

Nel problema di **Byzantine Agreement**, ogni nodo i riceve un input $b_i \in \{0, 1\}$ (non solo la sorgente). Il protocollo deve garantire:

1. **Consistency**: Per ogni coppia di nodi onesti i, j : $y_i = y_j$.
2. **Validity**: Se tutti i nodi onesti hanno lo stesso input b , allora $y_i = b$ per ogni nodo onesto i .

Le differenze rispetto al Byzantine Broadcast sono cruciali:

- Nel BA, *tutti* i nodi hanno un input (non solo la sorgente);
- La Validity in BA è più debole: richiede l'accordo sull'output solo quando tutti gli onesti concordano sull'input;
- Il BA richiede $f < n/2$ per essere risolvibile (vs. qualunque f per BB con PKI).

Osservazione — Lower Bound per BA: $f < n/2$

Il Byzantine Agreement non può essere risolto se $f \geq n/2$: con $n/2$ nodi corrotti, questi possono simulare due partizioni di $n/2$ nodi onesti che ricevono input diversi, forzando una violazione di Validity.

Senza PKI, il lower bound si abbassa a $f < n/3$ (stesso del BB), per un argomento di indistinguibilità analogo a quello visto per il BB.

9 Il Modello Asincrono e il Teorema FLP

9.1 Il Modello Asincrono

Nel modello asincrono non vi è né un clock globale né un bound sui tempi di comunicazione. Un protocollo è *event-driven*: specifica cosa fa un nodo quando riceve un messaggio.

Definizione — Configurazione, Messaggio, Schedule

- Una **configurazione** $C = (S, M)$ è una coppia dove S descrive lo stato di ogni nodo e M è la *message pool* (messaggi non ancora consegnati). Un messaggio $m = (p, x)$ specifica il nodo destinatario p e il contenuto x .
- Dato un messaggio $m \in M$, $m(C)$ indica la configurazione risultante dalla consegna di m al nodo p : lo stato di p si aggiorna, m viene rimosso da M e i nuovi messaggi inviati da p vengono aggiunti.
- Uno **schedule** $\sigma = (m_1, \dots, m_k)$ è una sequenza ordinata di messaggi. σ è *applicabile* a C se ogni m_i appartiene alla message pool della configurazione ottenuta applicando i primi $i - 1$ messaggi.

Approfondimento — Commutatività degli Schedule

Una proprietà fondamentale del modello asincrono è la **commutatività**: se due messaggi $m_1 = (p_1, x_1)$ e $m_2 = (p_2, x_2)$ hanno destinatari diversi ($p_1 \neq p_2$), allora

$$m_2(m_1(C)) = m_1(m_2(C))$$

perché l'elaborazione di un messaggio da parte di p_1 non influenza lo stato di p_2 prima che p_2 riceva il proprio messaggio. Più in generale, se due schedule σ_1 e σ_2 hanno insiemi di nodi destinatari disgiunti, allora $\sigma_2(\sigma_1(C)) = \sigma_1(\sigma_2(C))$.

Questa proprietà è il cuore tecnico della dimostrazione del teorema FLP.

9.2 Il Teorema FLP: Enunciato

Teorema FLP — Impossibilità del Consenso Asincrono

Teorema (Fischer, Lynch, Paterson, 1985): Nel modello asincrono, **nessun protocollo deterministico** per Byzantine Agreement che termina in tempo finito può soddisfare Consistency e Validity, se c'è almeno un nodo corrotto.

Questo è uno dei risultati più profondi dell'informatica distribuita. Nella sua forma originale, Fischer, Lynch e Paterson lo dimostrarono per il *consensus problem* con crash failures (non Byzantine), ma la tecnica si generalizza.

9.3 Struttura della Dimostrazione

La dimostrazione si basa sulla nozione di **valenza** di una configurazione:

Definizione — Configurazioni Univalenti e Bivalenti

Dato un protocollo deterministico Π_{ba} per BA:

- Una configurazione C è **0-valente** se, qualunque cosa faccia l'avversario, tutti i nodi onesti danno output 0.

- Una configurazione C è **1-valente** se, qualunque cosa faccia l'avversario, tutti i nodi onesti danno output 1.
- Una configurazione C è **bivalente** se non è né 0-valente né 1-valente (i.e., l'avversario può far sì che l'output sia 0 o 1).

La dimostrazione del teorema FLP si articola in due lemmi:

9.3.1 Lemma 1: Esistenza di una Configurazione Iniziale Bivalente

Lemma 9.1 (FLP, Lemma 1). *Se Π_{ba} è un protocollo deterministico per BA che termina e soddisfa Validity e Consistency, allora esiste un'assegnazione degli input ai nodi tale che la configurazione iniziale è bivalente.*

Dimostrazione. Per $i = 0, 1, \dots, n$, sia X_i la configurazione iniziale in cui i nodi $\{1, \dots, i\}$ hanno input 1 e i nodi $\{i+1, \dots, n\}$ hanno input 0. Per Validity: X_0 (tutti input 0) è 0-valente e X_n (tutti input 1) è 1-valente. Quindi esiste un indice i tale che X_{i-1} è 0-valente e X_i non è 0-valente. Mostriamo che X_i è bivalente.

Poiché Π_{ba} termina anche in presenza di un nodo corrotto, esiste uno schedule σ applicabile a X_{i-1} che non contiene messaggi per il nodo i e tale che $\sigma(X_{i-1})$ è una configurazione terminale per tutti i nodi diversi da i . Poiché X_{i-1} è 0-valente, $\sigma(X_{i-1})$ ha output 0 per tutti i nodi diversi da i .

Le configurazioni X_{i-1} e X_i differiscono solo per l'input del nodo i , e σ non contiene messaggi per i : quindi anche $\sigma(X_i)$ ha output 0 per tutti i nodi diversi da i . Dunque X_i non può essere 1-valente. Poiché per costruzione non è neanche 0-valente, è bivalente. $\square$

9.3.2 Lemma 2: Mantenimento della Bivalenza

Lemma 9.2 (FLP, Lemma 2). *Se Π_{ba} è un protocollo deterministico per BA che termina e soddisfa Validity e Consistency, C è bivalente e $m = (p, x) \in M$, allora esiste uno schedule $\sigma = (m_1, \dots, m_k)$ applicabile a C tale che $m_k = m$ e $\sigma(C)$ è bivalente.*

La dimostrazione di questo lemma è tecnica e usa la commutatività degli schedule. L'idea chiave è:

1. Si considera la configurazione $m(C)$. Se è bivalente, abbiamo finito.
2. Altrimenti, diciamo che $m(C)$ è 0-valente (WLOG). Si cercano le configurazioni raggiungibili da C senza consegnare m .
3. Tra queste, la prima configurazione Y che non è di tipo 0^* (cioè tale che $m(Y)$ è 0-valente) e la sua predecessora X sono il punto critico.
4. Si dimostra che $Z = m(Y)$ è bivalente usando la commutatività degli schedule e il fatto che Π_{ba} termina anche se il nodo p è corrotto.

9.3.3 Conclusione del Teorema FLP

Dimostrazione del Teorema FLP. Supponiamo per assurdo che esista un protocollo deterministico Π_{ba} che termina e soddisfa Validity e Consistency. Per il Lemma 1, esiste una configurazione iniziale bivalente C_0 . Per ogni $i \geq 1$, sia m_i il messaggio più vecchio in M_{i-1} , sia σ_i lo schedule garantito dal Lemma 2 (che termina con m_i) e sia $C_i = \sigma_i(C_{i-1})$ la configurazione bivalente risultante.

Lo schedule $(\sigma_1, \sigma_2, \sigma_3, \dots)$ è infinito, garantisce che ogni messaggio venga prima o poi consegnato (evitando che l'avversario possa “dimenticare” messaggi), e mantiene il protocollo in configurazioni bivalenti per sempre. Quindi il protocollo non termina mai, contraddicendo l'ipotesi. $\square$

Approfondimento — Implicazioni del Teorema FLP

Il teorema FLP ha implicazioni profonde per la progettazione di sistemi distribuiti:

- **Nessun sistema deterministico completamente asincrono** può garantire contemporaneamente safety (consistency), liveness (terminazione) e tolleranza ai guasti.
- Nella pratica, i sistemi distribuiti devono *scegliere* quale proprietà sacrificare. Il **teorema CAP** (Brewer, 2000) formalizza un trade-off simile per basi di dati distribuite (Consistency, Availability, Partition-tolerance).
- La via d'uscita è la **randomizzazione**: protocolli probabilistici possono aggirare il teorema FLP, garantendo terminazione con probabilità 1 pur soddisfacendo safety.

10 Un Protocollo Probabilistico per BA nel Modello Asincrono

10.1 Il Protocollo di Ben-Or (1983)

Michael Ben-Or propose nel 1983 il primo protocollo probabilistico per BA nel modello completamente asincrono. Il protocollo evita il teorema FLP perché la randomizzazione rompe la determinatezza che rende possibile la costruzione dello schedule avversariale infinito.

Protocollo di Ben-Or per BA Asincrono

Ogni nodo $u \in [n]$:

1. Riceve input $b \in \{0, 1\}$; inizializza $t = 1$; invia (b, t) a tutti; decided = False.
2. **While not decided:**
 - Quando arrivano $\geq n - f$ messaggi $(\hat{b}, t)$ nella fase t :
 - Conta v_0 (messaggi con $\hat{b} = 0$) e v_1 (messaggi con $\hat{b} = 1$).
 - decided = $(\max\{v_0, v_1\} \geq n/2 + 3f + 1)$.
 - Se $v_0 \geq n/2 + f + 1$: imposta $y = 0$.
 - Else se $v_1 \geq n/2 + f + 1$: imposta $y = 1$.
 - Else: sceglie $y \in \{0, 1\}$ uniformemente a caso.
 - $t = t + 1$; invia (y, t) a tutti.
3. Output y .

Teorema — Correttezza del Protocollo di Ben-Or

Se il numero di nodi corrotti è $f < (n - 2)/10$, il protocollo di Ben-Or:

1. Soddisfa **Validity** con probabilità 1;
2. Soddisfa **Consistency** (se termina);
3. **Termina** con probabilità 1.

Approfondimento — Analisi delle Soglie e Terminazione

Le soglie $n/2 + f + 1$ (per decisione deterministica) e $n/2 + 3f + 1$ (per terminazione) sono scelte con cura:

Soglia deterministica $n/2 + f + 1$: Se u imposta $y = 0$ deterministicamente, ha ricevuto $\geq n/2 + f + 1$ messaggi con valore 0. Allora nessun altro nodo onesto v può ricevere $\geq n/2 + f + 1$ messaggi con valore 1: il totale di messaggi sarebbe $> n + 2f$, impossibile con soli n nodi di cui f corrotti che possono inviare al più due valori.

Terminazione: Per ogni fase t in cui *decided* è False per tutti gli onesti, esiste $b \in \{0, 1\}$ tale che la probabilità che tutti i nodi onesti inviino $(b, t + 1)$ è almeno 2^{-n} . Dunque la probabilità di non terminare dopo t fasi è al più $(1 - 2^{-n})^t \rightarrow 0$.

Costo: Il prezzo della randomizzazione è l'efficienza: nel caso peggiore, il protocollo richiede un numero esponenziale di fasi prima di terminare. Protocolli successivi (Feldman-Micali, Cachin-Kursawe-Shoup) usano la crittografia per ridurre il numero atteso di fasi a una costante.

11 Il Protocollo di Consenso di Bitcoin: Nakamoto Consensus**11.1 Il Cambiamento di Paradigma: da Permissioned a Permissionless****Nota Storica — Bitcoin e il White Paper di Nakamoto (2008)**

Nel 2008, un autore pseudonimo noto come Satoshi Nakamoto pubblicò il white paper *Bitcoin: A Peer-to-Peer Electronic Cash System*, proponendo un sistema di pagamento digitale senza autorità centrale. La chiave dell'innovazione non era singola: Bitcoin combinava proof-of-work (già proposta da Back nel 2002 con Hashcash, e concettualmente da Dwork e Naor nel 1993), strutture a catena di hash, e incentivi economici in un sistema coerente e funzionante. Il 3 gennaio 2009, il *genesis block* di Bitcoin fu estratto, dando inizio alla prima blockchain operativa su larga scala.

In un ambiente permissionless, il principale problema è prevenire i **Sybil attack**: un singolo agente che crea molte identità per dominare il sistema. La soluzione di Bitcoin lega la “voce in capitolo” a risorse computazionali reali.

11.2 Proof-of-Work: Definizione e Funzione**Definizione — Proof-of-Work**

Una **proof-of-work** è un dato π che:

- è **difficile da trovare**: richiede in media un numero $W = 2^{256}/\text{target}$ valutazioni di una funzione hash crittografica;
- è **facile da verificare**: richiede una sola valutazione della funzione hash.

Nel contesto di Bitcoin, la proof-of-work per un blocco con header h rispetto a un target T è un nonce η tale che $\text{sha256}(\text{sha256}(h)) < T$.

L'intuizione è che la proof-of-work trasforma il potere computazionale in “voti”: chi ha più capacità di calcolo produce più blocchi, e la catena più lunga (in termini di lavoro cumulativo) è considerata valida da tutti.

11.3 Struttura della Blockchain di Bitcoin

11.3.1 La Struttura dell'Header

Ogni blocco nella blockchain di Bitcoin ha uno header di esattamente **80 byte**, composto dai seguenti campi:

Campo	Dimensione	Descrizione
version	4 byte	Versione del protocollo
prev_hash	32 byte	SHA256 dello header del blocco precedente
data_hash	32 byte	SHA256 (Merkle root) delle transazioni
timestamp	4 byte	Unix time della creazione del blocco
target	4 byte	Target (in notazione scientifica custom)
nonce	4 byte	Numero arbitrario per soddisfare il target

La struttura incatenata degli hash (ogni blocco contiene l'hash del precedente) rende computazionalmente proibitivo modificare un blocco passato: occorrerebbe ricalcolare i nonce di tutti i blocchi successivi.

11.3.2 Le Proprietà di Eventual Consistency e Liveness

Proprietà del Nakamoto Consensus

Il protocollo di consenso di Nakamoto garantisce (sotto opportune assunzioni):

- **Eventual Consistency:** Le copie locali della blockchain dei nodi onesti concordano, eccetto al più per un numero limitato di blocchi “recenti” in coda.
- **Liveness:** Se un nodo onesto riceve una transazione valida, questa viene prima o poi inclusa in un blocco della catena principale.
- **Resistenza ai Sybil attack:** Un nodo con il $k\%$ della capacità computazionale complessiva produce in media il $k\%$ dei blocchi, indipendentemente da quante identità possieda.

11.4 Il Meccanismo di Aggiustamento del Target

Approfondimento — Il Difficulty Adjustment in Bitcoin

Bitcoin si propone di produrre, in media, un blocco ogni 10 minuti, indipendentemente dalla capacità computazionale complessiva della rete (l'*hash rate*). Per realizzare questo obiettivo, ogni 2016 blocchi (corrispondenti a circa 14 giorni a ritmo ideale) il target viene ricalcolato:

$$\text{target}(k) = \text{target}(k-1) \cdot \frac{\delta}{14 \text{ giorni}}$$

dove δ è la differenza effettiva di timestamp tra il primo e l'ultimo blocco dell'epoca appena conclusa. Se la rete ha prodotto i 2016 blocchi più velocemente di 14 giorni ($\delta < 14$ giorni), il target viene ridotto (rendendo il problema più difficile); se più lentamente, il target viene aumentato.

Rappresentazione del target: Il target è un intero positivo di 256 bit, ma viene memorizzato negli header in soli 4 byte usando una rappresentazione in notazione scientifica custom: $\text{target} = \text{coeff} \times 2^{\text{exp}}$, dove **coeff** occupa i primi 3 byte e **exp** l'ultimo byte.

Stima dell'hash rate: Se il target attuale è T , la probabilità che un singolo hash soddisfi il requisito è $T/2^{256}$. Il numero atteso di hash per trovare un nonce valido è $2^{256}/T$. Se i blocchi vengono prodotti in media ogni 10 minuti, l'hash rate complessivo è circa:

$$h = \frac{2^{256}/T}{600} \text{ hash/secondo}$$

11.5 La Regola della Catena Più Lunga

Un aspetto cruciale del protocollo di Nakamoto è la risoluzione dei conflitti tra catene alternative (*fork*). Quando un nodo riceve un blocco che estende una catena diversa da quella che sta seguendo, adotta la catena con la **maggiore proof-of-work cumulativa**:

$$\text{work}(\text{block}_{1:t}) = \sum_{i=1}^t \frac{2^{256}}{\text{block}[i].\text{header.target}}$$

Questo meccanismo garantisce che la catena “corretta” (quella prodotta dalla maggioranza onesta) cresca più velocemente di qualunque catena alternativa mantenuta da un avversario con meno del 50% della capacità computazionale.

Attenzione — Il 51% Attack

Se un avversario controlla più del 50% dell'hash rate totale, può teoricamente:

- Produrre una catena alternativa più lunga di quella onesta;
- Riscrivere la storia delle transazioni (double spending);
- Escludere alcune transazioni dalla blockchain.

Questo è il **51% attack**, il principale vettore di attacco al Nakamoto Consensus. La sicurezza del sistema dipende dall'assunzione che la maggioranza dell'hash rate sia controllata da nodi onesti.

Il costo economico di un tale attacco — l'hardware e l'energia necessari — è il principale deterrente nella pratica. Per la blockchain di Bitcoin, al momento attuale, il costo sarebbe nell'ordine di decine di miliardi di euro.

12 Le Transazioni Bitcoin

12.1 Struttura di Base

In Bitcoin non esistono “monete” digitali come oggetti fisici: ciò che esiste sono le **transazioni**. Una transazione specifica un insieme di *input* (fondi in ingresso) e un insieme di *output* (fondi in uscita).

Definizione — Struttura di una Transazione Bitcoin

Una transazione tx è composta da:

- **Version** (4 byte): numero di versione del formato.
- **Inputs:** Lista di zero o più input. Ogni input contiene:
 - Un puntatore (*prev_tx*, *prev_id*) all'output di una transazione precedente;
 - Un *unlocking script* (*scriptSig*);

- Un *sequence number*.
- **Outputs:** Lista di uno o più output. Ogni output contiene:
 - Un valore in satoshi (1 satoshi = 10^{-8} BTC);
 - Un *locking script* (scriptPk).
- **Locktime** (4 byte): specifica il momento a partire dal quale la transazione può essere inclusa in un blocco.

12.2 Il Modello UTXO

Definizione — UTXO (Unspent Transaction Output)

Un output di una transazione è **non speso** (UTXO, Unspent Transaction Output) se non è ancora stato usato come input di un'altra transazione. L'insieme di tutti gli UTXO costituisce il **UTXO set**, che rappresenta l'insieme di tutti i bitcoin "in circolazione". Il saldo di un indirizzo Bitcoin è la somma dei valori di tutti gli UTXO associati a quell'indirizzo.

12.3 Transazione Coinbase e Creazione di Bitcoin

La Coinbase Transaction e il Dimezzamento

La prima transazione di ogni blocco è la **coinbase transaction**: non ha input (o ha un input speciale) e crea nuovi bitcoin. L'ammontare creato per il blocco i -esimo è:

$$\text{new_btc}[i] = \frac{50}{2^{\lfloor i/210000 \rfloor}} \text{ BTC}$$

Ogni 210000 blocchi (circa 4 anni), il reward si dimezza (*halving*). La somma geometrica garantisce che il totale di bitcoin in circolazione non supererà mai 21 milioni:

$$\sum_{k=0}^{\infty} 210000 \cdot \frac{50}{2^k} = 210000 \cdot 50 \cdot 2 = 21,000,000 \text{ BTC}$$

Considerando che l'unità minima è 1 satoshi (10^{-8} BTC), i nuovi blocchi smetteranno di generare valuta intorno all'anno 2140.

12.4 Validità di una Transazione

Una transazione tx è **valida** se:

1. Tutti gli input puntano a UTXO non ancora spesi;
2. Per ogni input, la concatenazione di unlocking script e locking script dell'output puntato restituisce **True** (vedremo il dettaglio nella sezione successiva);
3. La somma dei valori degli output $\leq$ la somma dei valori degli UTXO puntati dagli input;
4. La differenza (non negativa) è la **transaction fee**:

$$\text{tx.fee} = \sum_{\text{input}} \text{input}(\text{prev_tx}, \text{prev_id}).\text{valore} - \sum_{\text{output}} \text{output}.\text{valore} \geq 0$$

13 Il Linguaggio Script di Bitcoin

13.1 Architettura del Linguaggio

Il linguaggio **Script** è un semplice linguaggio di programmazione *stack-based* e *stateless* utilizzato per definire le condizioni di spesa degli output Bitcoin. Ogni nodo della rete contiene un interprete Script.

Definizione — Modello di Esecuzione di Script

Uno script è una sequenza di *comandi*, che possono essere:

- **Dati:** sequenze di byte inserite direttamente nello stack;
- **Op code:** istruzioni che manipolano lo stack (estraggono, elaborano e reintroducono elementi).

L'esecuzione di uno script per la validazione di un input avviene concatenando:

[unlocking_script] + [locking_script]

Lo script restituisce **True** se e solo se al termine dell'esecuzione lo stack contiene un unico elemento non nullo.

13.2 Tipi Principali di Locking Script

13.2.1 Pay-to-Public-Key (P2PK)

Il tipo più semplice:

- **Locking script:** <pk> OP_CHECKSIG
- **Unlocking script:** <sig>
- **Verifica:** OP_CHECKSIG verifica che <sig> sia una firma valida della transazione rispetto a <pk>.

13.2.2 Pay-to-Public-Key-Hash (P2PKH)

Esempio Applicativo — P2PKH

Il P2PKH aggiunge un livello di protezione: la chiave pubblica non è esposta fino al momento della spesa.

- **Locking script:** OP_DUP OP_HASH160 <pkhash> OP_EQUALVERIFY OP_CHECKSIG
- **Unlocking script:** <sig> <pk>
- **Esecuzione:**
 1. <sig> e <pk> vengono messi nello stack;
 2. OP_DUP duplica <pk>;
 3. OP_HASH160 calcola `hash160(<pk>)`;
 4. OP_EQUALVERIFY verifica che `hash160(<pk>) = <pkhash>`;
 5. OP_CHECKSIG verifica la firma.

Vantaggi: la chiave pubblica è protetta dall'hash fino al momento della spesa, fornendo un ulteriore livello di sicurezza contro futuri attacchi al logaritmo discreto su curve ellittiche (ECDLP).

13.2.3 Pay-to-Multisig (P2MS)

Il P2MS permette di richiedere t firme su n per spendere un output (*threshold signature*):

- **Locking script** (esempio 2-di-3): `OP_2 <pkA> <pkB> <pkC> OP_3 OP_CHECKMULTISIG`
- **Unlocking script**: `OP_0 <sigB> <sigC>`

Nota: il `OP_0` all'inizio dell'unlocking script è necessario a causa di un bug storico nell'implementazione originale di `OP_CHECKMULTISIG`, che estrae un elemento extra dallo stack.

13.2.4 Pay-to-Script-Hash (P2SH)

Approfondimento — P2SH e la Separazione degli Oneri

Il P2SH (introdotto con BIP-16 nel 2012) inverte la responsabilità di specificare le condizioni di spesa:

- Il **destinatario** crea il *redeem script* con le condizioni di spesa e invia al mittente solo il suo hash.
- Il **mittente** include l'hash nel locking script: `OP_HASH160 <redeem_script_hash> OP_EQUAL`
- Quando il destinatario vuole spendere, include il redeem script nell'unlocking script.

Vantaggi:

- Il mittente non deve conoscere le condizioni di spesa (utile per pagare a un wallet multisig senza conoscerne la struttura);
- Gli script complessi non appesantiscono la blockchain fino al momento della spesa;
- Standardizzazione: l'UTXO set contiene solo hash di 20 byte invece di script potenzialmente lunghi.

13.2.5 OP_RETURN e Dati nella Blockchain

L'istruzione `OP_RETURN` permette di inserire dati arbitrari nella blockchain in modo “pulito”: lo script fallisce immediatamente, rendendo l'output **non spendibile** e quindi non mantenuto nell'UTXO set. Questo meccanismo è usato per inserire metadati (timestamp di documenti, dati di protocolli di layer 2, ecc.) senza “inquinare” l'UTXO set con output permanentemente non spendibili.

14 Crittografia basata su Curve Ellittiche (ECC)

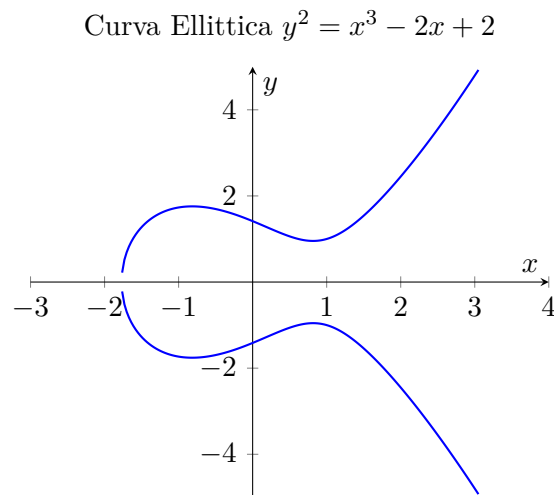
14.1 Definizione e Proprietà Matematiche

Possiamo descrivere una curva ellittica come l'insieme dei punti (x, y) , a coordinate in un opportuno campo, che soddisfano l'equazione:

$$y^2 = x^3 + ax + b \quad (1)$$

dove a e b sono parametri che definiscono la curva, con l'aggiunta di un punto speciale che chiamiamo *punto all'infinito* ∞ .

Di seguito un esempio visivo della curva sui numeri reali $\mathbb{R}$. Il grafico illustra la forma tipica della curva al variare di x :



14.2 Campi Finiti e la Curva SECP256K1

Per le applicazioni in crittografia non si considera il campo dei numeri reali, ma i campi finiti $\mathbb{F}_p$ dove p è un numero primo enorme. Questo significa che l'insieme delle coordinate è limitato agli interi $\{0, 1, \dots, p-1\}$ con operazioni modulo p .

La curva ellittica che viene usata in Bitcoin si chiama **secp256k1** ed è definita dai parametri:

$$a = 0, \quad b = 7$$

$$p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$$

14.3 Somma di Punti e Moltiplicazione Scalare

Su una curva ellittica $\mathcal{C} = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\infty\}$ si definisce un'operazione di *somma* geometrica tra due punti $P, Q \in \mathcal{C}$ che restituisce un terzo punto in $\mathcal{C}$. Questa operazione rende $(\mathcal{C}, +)$ un gruppo.

Dato un punto $G \in \mathcal{C}$ e un intero $k \in \mathbb{N}$, possiamo calcolare la moltiplicazione scalare:

$$P = kG = \underbrace{G + G + \dots + G}_{k \text{ volte}}$$

Siccome il gruppo permette il calcolo di $P + P$ (point doubling) in tempo $\mathcal{O}(1)$, è possibile calcolare kG in tempo $\mathcal{O}(\log k)$ sfruttando algoritmi come il *double-and-add*.

14.4 Discrete Logarithm Problem e ECDH

La sicurezza della crittografia su curve ellittiche (ECC) si basa sul **Discrete Logarithm Problem (DLP)** su curve ellittiche:

INPUT: Due punti sulla curva $G, P \in \mathcal{C}$

OUTPUT: Un intero $k \in \mathbb{N}$ tale che $P = kG$, oppure **None** se tale intero non esiste.

Al momento, nessuno conosce un algoritmo in tempo polinomiale per risolvere questo problema.

Dato un *punto base* generatore G , si definisce una coppia di chiavi:

- **Chiave Segreta (sk):** Un intero positivo scelto casualmente.
- **Chiave Pubblica (pk):** Il punto sulla curva calcolato come $pk = sk \cdot G$.

Accenno a Diffie-Hellman (ECDH): La difficoltà del DLP permette a due entità (Alice e Bob) di concordare una chiave crittografica condivisa su un canale insicuro. Se Alice ha (sk_A, pk_A) e Bob ha (sk_B, pk_B) , Alice può calcolare $K = sk_A \cdot pk_B$ e Bob calcola $K = sk_B \cdot pk_A$. Grazie alla commutatività della moltiplicazione scalare, entrambi ottengono:

$$K = sk_A \cdot (sk_B \cdot G) = sk_B \cdot (sk_A \cdot G)$$

Un intercettatore che osserva solo i punti pubblici pk_A e pk_B non può determinare K senza risolvere il logaritmo discreto.

15 Sintesi e Connessioni Interdisciplinari

15.1 Il Quadro d'Insieme

Ripercorrendo il percorso del corso, possiamo ora apprezzare la coerenza del quadro teorico:

Contesto	Risultato	Implicazione
Sincrono + PKI	Dolev-Strong funziona	BB risolubile per qualunque f
Sincrono, senza PKI	Lower bound $f < n/3$	BB impossibile per $f \geq n/3$
Sincrono, senza PKI, $f < n/3$	Protocollo randomizzato	BB con high probability
Asincrono, deterministico	Teorema FLP	BA impossibile con ≥ 1 corrotto
Asincrono, probabilistico	Ben-Or	BA con prob. 1 per $f < (n - 2)/10$
Permissionless	Nakamoto Consensus	Consenso con proof-of-work

15.2 Connessioni con Altre Discipline

Approfondimento — Connessioni Interdisciplinari

Lo studio delle criptovalute è intrinsecamente interdisciplinare:

- **Teoria dei Giochi:** Il protocollo di Bitcoin può essere analizzato come un gioco strategico tra miner. La ricerca sull'equilibrio di Nash per le strategie di mining (selfish mining, pool mining) è un'area attiva.
- **Crittografia:** Firme digitali (RSA, ECDSA), funzioni hash crittografiche, schemi di commitment, e la nozione di sicurezza computazionale sono fondamentali.
- **Economia:** La politica monetaria di Bitcoin (supply fissa, halving) e i meccanismi di incentivo (block reward + transaction fee) sono scelte di design economico deliberate.
- **Teoria della Complessità:** La difficoltà computazionale dei problemi su cui si basano le primitive crittografiche (fattorizzazione, logaritmo discreto, SHA256 preimage) è un assunto fondamentale.
- **Reti e Sistemi Distribuiti:** La propagazione delle transazioni e dei blocchi nella rete P2P, il meccanismo di gossip, la gestione dei fork, sono problemi di networking.

Riferimenti Bibliografici

Riferimenti bibliografici

- [1] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Whitepaper*, 2008.
- [2] D. Dolev and H. R. Strong. Authenticated algorithms for byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.
- [3] L. Lamport, R. Shostak, and M. Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, pages 382–401, July 1982.
- [4] M. Pease, R. Shostak, and L. Lamport. Reaching agreement in the presence of faults. *Journal of the ACM*, 27(2):228–234, 1980.
- [5] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
- [6] M. J. Fischer, N. A. Lynch, and M. Merritt. Easy impossibility proofs for distributed consensus problems. *Distributed Computing*, 1:26–39, 1986.
- [7] M. Ben-Or. Another advantage of free choice (extended abstract): completely asynchronous agreement protocols. In *Proceedings of the 2nd ACM PODC*, pages 27–30, 1983.
- [8] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [9] J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. Chapman and Hall/CRC, 2007.
- [10] M. Bellare and P. Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM CCS*, pages 62–73, 1993.
- [11] A. Back. Hashcash: A denial of service counter-measure. *Whitepaper*, 2002.
- [12] C. Dwork and M. Naor. Pricing via processing or combatting junk mail. In *CRYPTO '92*, pages 139–147. Springer, 1992.
- [13] E. Shi. *Foundations of Distributed Consensus and Blockchains*. Available at <https://www.distributedconsensus.net>, 2020.
- [14] P. Feldman and S. Micali. Optimal algorithms for byzantine agreement. In *Proceedings of the 20th ACM STOC*, pages 148–161, 1988.
- [15] C. Cachin, K. Kursawe, and V. Shoup. Random oracles in constantipole: Practical asynchronous byzantine agreement using cryptography. In *Proceedings of the 19th ACM PODC*, pages 123–132, 2000.
- [16] T. P. Pedersen. A threshold cryptosystem without a trusted party. In *EUROCRYPT '91*, pages 522–526. Springer, 1991.